Summarize

Chapter 13. Scaling

> *"You're gonna need a bigger boat."*
>
> —Chief Brody, *Jaws*

When we scale our systems, we do so for one of two reasons. Firstly, it allows us to improve the performance of our system, perhaps by allowing us to handle more load or by improving latency. Secondly, we can scale our system to improve its robustness. In this chapter, we'll look at a model to describe the different types of scaling, and then we'll look in detail at how each type of scaling can be implemented using a microservice architecture. At the end of this chapter, you should have an array of techniques to handle the scaling issues that may come your way.

To start with, though, let's look at the different types of scaling you might want to apply.

The Four Axes of Scaling

There isn't one right way to scale a system, as the technique used will depend on the type of constraint you might have. We have a number of different types of scaling we can bring to bear to help with performance, robustness, or perhaps both. A model I've often used to describe the different types of scaling is the Scale Cube from *The Art of Scalability*,[1] which breaks scaling down into three categories that in the context of computer systems cover functional decomposition, horizontal duplication, and data partitioning. The value of this model is that it helps you understand that you can scale a system along one, two, or all three of these axes, depending on your needs. However, especially in a world of virtualized infrastructure, I've always felt that this model lacked a fourth axis of vertical scaling, though that would have the somewhat unfortunate property of making it no longer a cube. Nonetheless, I think it's a useful set of mechanisms for us to determine how best to scale our microservice architectures. Before we look at

these types of scaling in detail, along with the relative pros and cons, a brief summary is in order:

Vertical scaling

: In a nutshell, this means getting a bigger machine.

Horizontal duplication

: Having multiple things capable of doing the same work.

Data partitioning

: Dividing work based on some attribute of the data, e.g., customer group.

Functional decomposition

: Separation of work based on the type, e.g., microservice decomposition.

Understanding what combination of these scaling techniques is most appropriate will fundamentally come down to the nature of the scaling issue you are facing. To explore this in more detail, as well as look at examples of how these concepts could be implemented for MusicCorp, we'll also examine their suitability for a real-world company, FoodCo.[2] FoodCo provides food delivery direct to customers across a number of countries around the world.

Vertical Scaling

Some operations can just benefit from more grunt. Getting a bigger box with faster CPU and better I/O can often improve latency and throughput, allowing you to process more work in less time. So if your application isn't going fast enough for you or can't handle enough requests, why not get a bigger machine?

In the case of FoodCo, one of the challenges it is suffering from is increasing write contention on its primary database. Normally, vertical scaling is an absolute go-to option to quickly scale writes on a relational database, and indeed FoodCo has already upgraded the database infrastructure multiple times. The problem is that FoodCo has really already pushed this as far as it is comfortable with. Vertical scaling worked for many years, but given the company's growth projections, even if FoodCo could get a bigger machine, that isn't likely to solve the problem in the longer term.

Historically, when vertical scaling required purchasing hardware, this technique was more problematic. The lead time to buying hardware meant that this wasn't something

that could be entered into lightly, and if it turned out that having a bigger machine didn't solve your problems, then you had likely spent a bunch of money you didn't need to. In addition, it was typical to oversize the machine you needed due to the hassle of getting budgetary sign-offs, waiting for the machine to arrive and so on, which in turn led to significant unused capacity in data centers.

The move to virtualization and the emergence of the public cloud have helped immensely with this form of scaling, though.

Implementation

Implementation will vary depending on whose infrastructure you are running on. If running on your own virtualized infrastructure, you may be able to just resize the VM to use more of the underlying hardware—this is something that should be quick and fairly risk-free to implement. If the VM is as big as the underlying hardware can handle, this option is of course a nonstarter—you may have to buy more hardware. Likewise, if you're running on your own bare metal servers and don't have any spare hardware lying around that is bigger than what you are currently running on, then again, you're looking at having to buy more machines.

In general, if I've gotten to the point where I'd have to purchase new infrastructure to try out vertical scaling, due to the increased cost (and time) for this to have an impact I might well skip this form of scaling for the moment and look instead at horizontal duplication, which we'll come to next.

But the emergence of the public cloud has also allowed us to easily rent, on a per-hour basis (and in some cases on an even shorter-term basis), fully managed machines through public cloud vendors. Moreover, the main cloud providers offer a wider variety of machines for different types of problems. Is your workload more memory intensive? Treat yourself to an AWS `u-24tb1.metal` instance, which provides 24 TB of memory (yes, you read that right). Now, the number of workloads that might actually need this much memory seem pretty rare, but you have that option. You also have machines tailored to high I/O, CPU, or GPU uses as well. If your existing solution is already on the public cloud, this is such a trivial form of scaling to try that if you're after a quick win, it's a bit of a no-brainer.

Key benefits

On virtualized infrastructure, especially on a public cloud provider, implementing this form of scaling will be *fast*. A lot of the work around scaling applications comes down to

experimentation—having an idea about something that can improve your system, making the change, and measuring the impact. Activities that are quick and fairly risk-free to try out are always worth doing early on. And vertical scaling fits the bill here.

It's also worth noting that vertical scaling can make it easier to perform other types of scaling. As a concrete example, moving your database infrastructure to a larger machine may allow it to host the logically isolated databases for newly created microservices as part of functional decomposition.

Your code or database is unlikely to need any changes to make use of the larger underlying infrastructure, assuming the operating system and chipsets remain the same. Even if changes are needed to your application to make use of the change of hardware, they might be limited to things like increasing the amount of memory available to your runtime through runtime flags.

Limitations

As we scale up the machines we are running on, our CPUs often don't actually get faster; we just have more cores. This has been a shift over the last 5–10 years. It used to be that each new generation of hardware would deliver big improvements in CPU clock speed, meaning our programs got big jumps in performance. Clock speed improvements have drastically trailed off, though, and instead we get more and more CPU cores to play with. The problem is that often our software has not been written to take advantage of multicore hardware. This could mean that the shift of your application from a 4- to 8-core system may deliver little if any improvement, even if your existing system is CPU-bound. Changing the code to take advantage of multicore hardware can be a significant undertaking and can require a complete shift in programming idioms.

Having a bigger machine is also likely to do little to improve robustness. A larger, newer server might have improved reliability, but ultimately if that machine is down, then the machine is down. Unlike the other forms of scaling we'll look at, vertical scaling is unlikely to have much impact in improving your system's robustness.

Finally, as the machines get larger, they get more expensive—but not always in a way that is matched by the increased resources available to you. Sometimes this means it can be more cost effective to have a larger number of small machines, rather than a smaller number of large machines.

Horizontal Duplication

With horizontal duplication, you duplicate part of your system to handle more workloads. The exact mechanisms vary—we'll look at implementations shortly—but fundamentally horizontal duplication requires you to have a way of distributing the work across these duplicates.

As with vertical scaling, this type of scaling is on the simpler end of the spectrum and is often one of the things I'll try early on. If your monolithic system can't handle the load, spin up multiple copies of it and see if that helps!

Implementations

Probably the most obvious form of horizontal duplication that comes to mind is making use of a load balancer to distribute requests across multiple copies of your functionality, as we see in Figure 13-1, where we are load balancing across multiple instances of MusicCorp's `Catalog` microservice. Load balancer capabilities differ, but you'd expect them all to have some mechanism to distribute load across the nodes, and to detect when a node is unavailable and remove it from the load balancer pool. From the consumer's point of view, the load balancer is an entirely transparent implementation concern—we can view it as part of the logical boundary of the microservice in this regard. Historically, load balancers would be thought of primarily in terms of dedicated hardware, but this has long since ceased to be common—instead, more load balancing is done in software, often running on the client side.

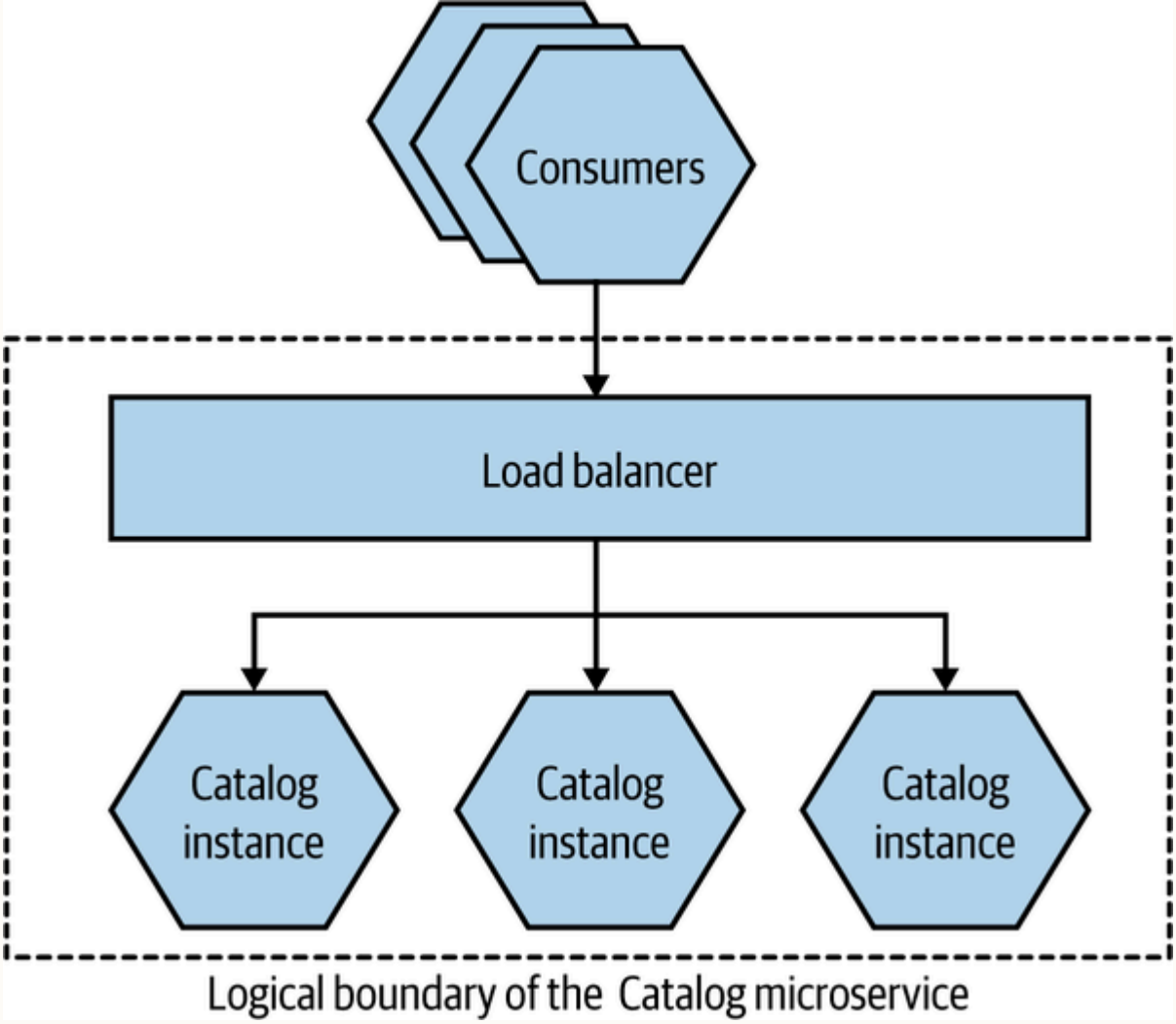


Figure 13-1. The `Catalog` microservice deployed as multiple instances, with a load balancer to spread requests

Another example of horizontal duplication could be the competing consumer pattern, as detailed in *Enterprise Integration Patterns*.[3] In Figure 13-2, we see new songs being uploaded to MusicCorp. These new songs need to be transcoded into different files to be used as part of MusicCorp's new streaming offering. We have a common queue of work in which these jobs are placed, and a set of `Song Transcoder` instances all consume from the queue—the different instances are competing over the jobs. To increase the throughput of the system, we can increase the number of `Song Transcoder` instances.

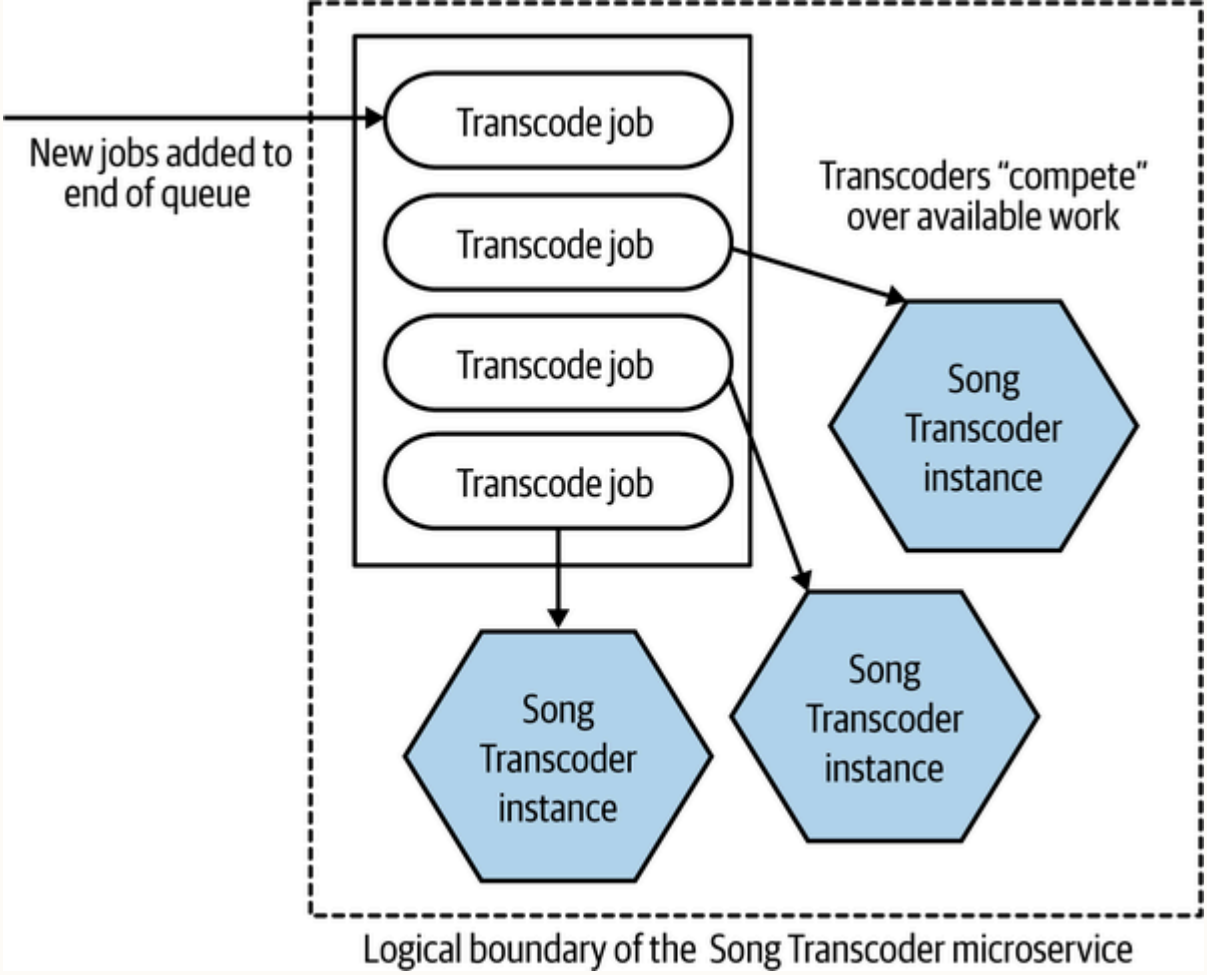


Figure 13-2. Transcoding for streaming being scaled up using the competing consumer pattern

In the case of FoodCo, a form of horizontal duplication has been used to reduce the read load on the primary database through the use of read replicas, as we see in Figure 13-3. This has reduced read load on the primary database node, freeing up resources to handle writes, and has worked very effectively, as a lot of the load on the main system was read-heavy. These reads could easily be redirected to these read replicas, and it's common to use a load balancer over multiple read replicas.

The routing to either the primary database or a read replica is handled internally in the microservice. It's transparent to consumers of this microservice as to whether or not a request they send ends up hitting the primary or read replica database.

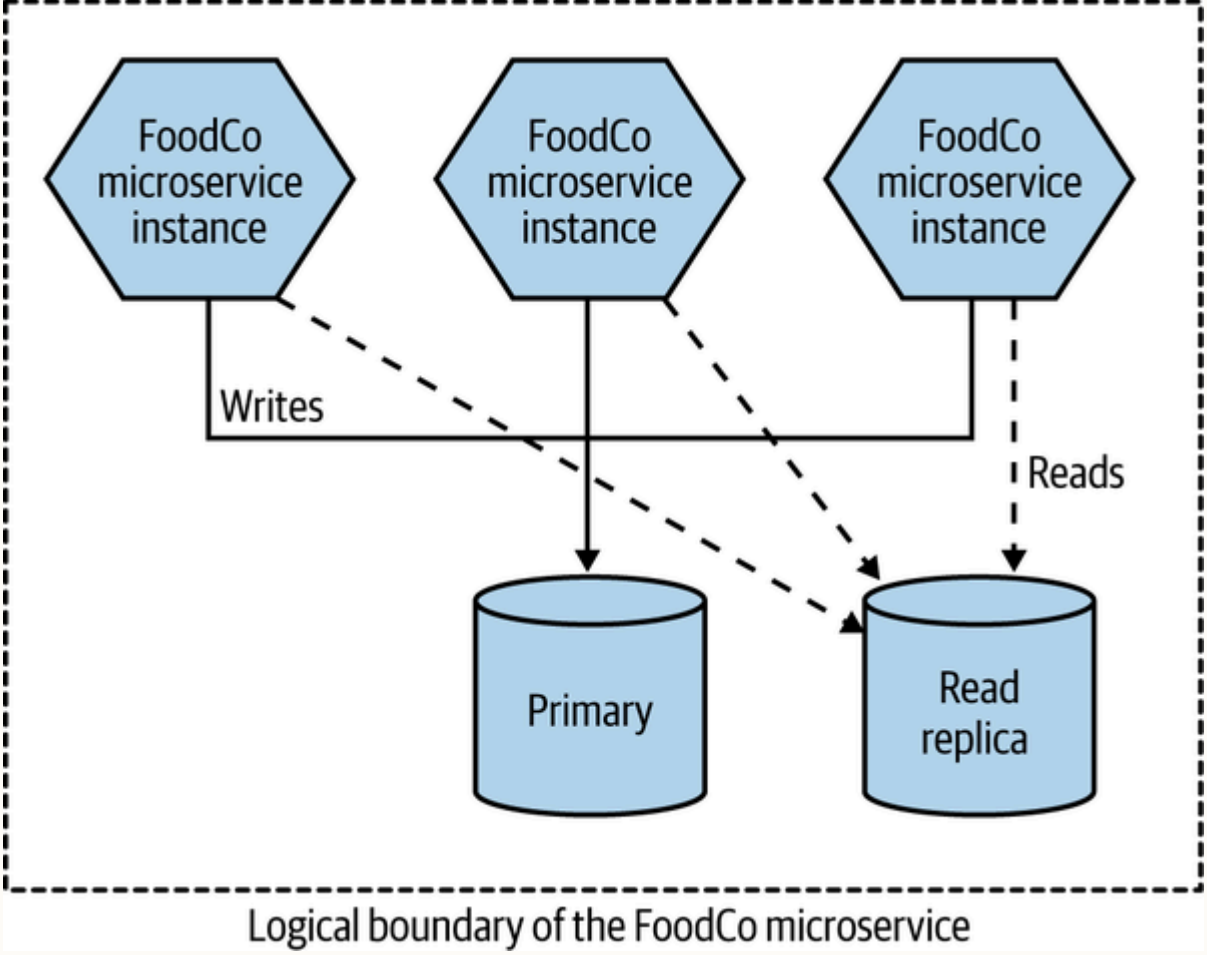


Figure 13-3. FoodCo making use of read replicas to scale read traffic

Key benefits

Horizontal duplication is relatively straightforward. It's rare that the application needs to be updated, as the work to distribute the load can often be done elsewhere—for example, through a queue running on a message broker or perhaps in a load balancer. If vertical scaling isn't available to me, this form of scaling is typically the next thing I'd look at.

Assuming the work can be easily spread across the duplicates, it's an elegant way of spreading the load and reducing contention for raw computing resources.

Limitations

As with virtually all the scaling options we'll look at, horizontal duplication requires more infrastructure, which can of course cost more money. It can be a bit of a blunt instrument too—you might run multiple full copies of your monolithic application, for example, even if only part of that monolith is actually experiencing scaling issues.

Much of the work here is in implementing your load distribution mechanisms. These can range from the simple, such as HTTP load balancing, to the more complex, such as using a message broker or configuring database read replicas. You are relying on this load dis-

tribution mechanism to do its job—coming to grips with how it works and with any limitations of your specific choice will be key.

Some systems might place additional requirements on the load distribution mechanism. For example, they might require that each request associated with the same user session gets directed to the same replica. This can be solved through the use of a load balancer that requires sticky session load balancing, but that might in turn limit what load distribution mechanisms you could consider. It's worth noting that systems that require sticky load balancing like this are prone to other problems, and in general I'd avoid building systems that have this requirement.

Data Partitioning

Having begun with the simpler forms of scaling, we're now entering into more difficult territory. Data partitioning requires that we distribute load based on some aspect of data—perhaps distributing load based on the user, for example.

Implementation

The way data partitioning works is that we take a key associated with the workload and apply a function to it, and the result is the *partition* (sometimes called a shard) we will distribute the work to. In Figure 13-4, we have two partitions, and our function is quite simple—we send the request to one database if the family name starts with A to M, or to a different database if the family name starts with N to Z. Now, this is actually a bad example of a partitioning algorithm (we'll get to why that is shortly), but hopefully this is straightforward enough to illustrate the idea.

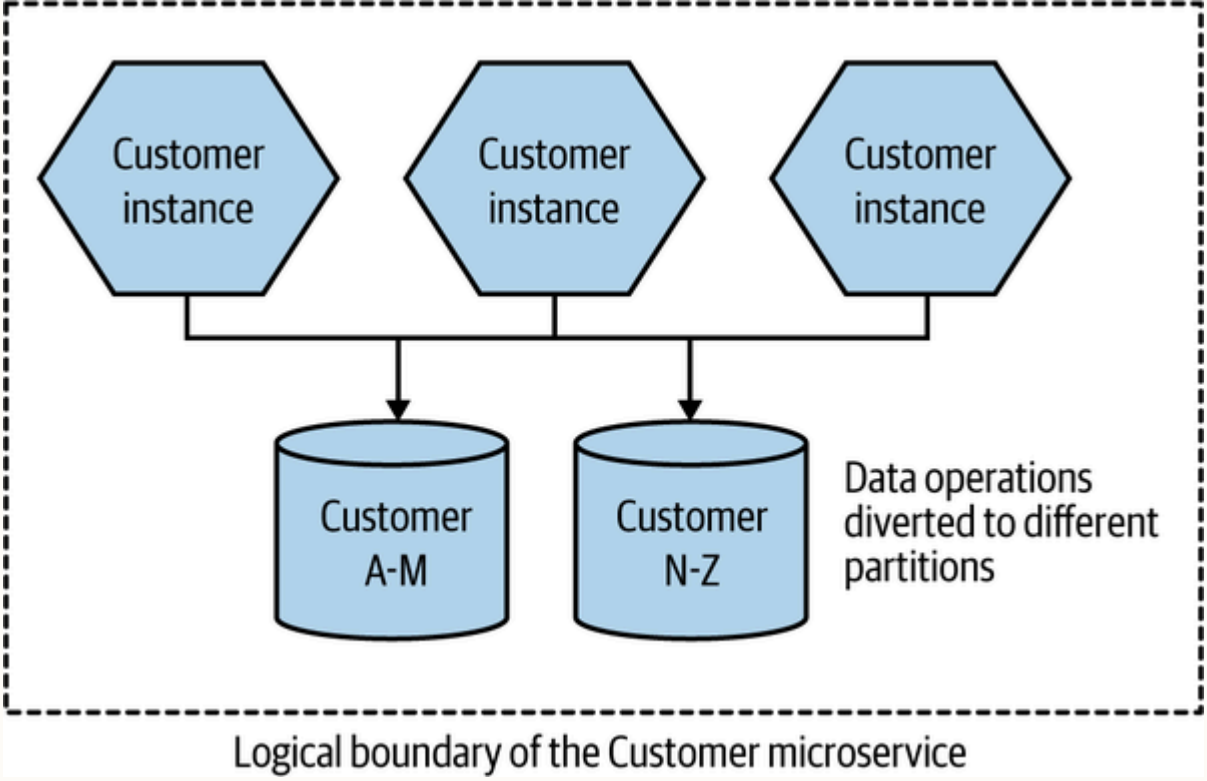


Figure 13-4. Customer data is partitioned across two different databases

In this example, we're partitioning at the database level. A request to the `Customer` microservice can hit any microservice instance. But when we perform an operation that requires the database (reads or writes), that request is directed to the appropriate database node based on the name of the customer. In the case of a relational database, the schema of both database nodes would be identical, but the contents of each would apply only to a subset of the customers.

Partitioning at the database level often makes sense if the database technology you are using supports this concept natively, as you can then offload this problem to an existing implementation. We could, however, partition instead at the microservice instance level, as we see in Figure 13-5. Here, we need to be able to work out from the inbound request what partition the request should be mapped to—in our example, this is being done via some form of proxy. In the case of our customer-based partitioning model, if the name of the customer is in the request headers, then that would be sufficient. This approach makes sense if you want dedicated microservice instances for partitioning, which could be useful if you are making use of in-memory caching. It also means you are able to scale each partition at both the database level and the microservice instance level.

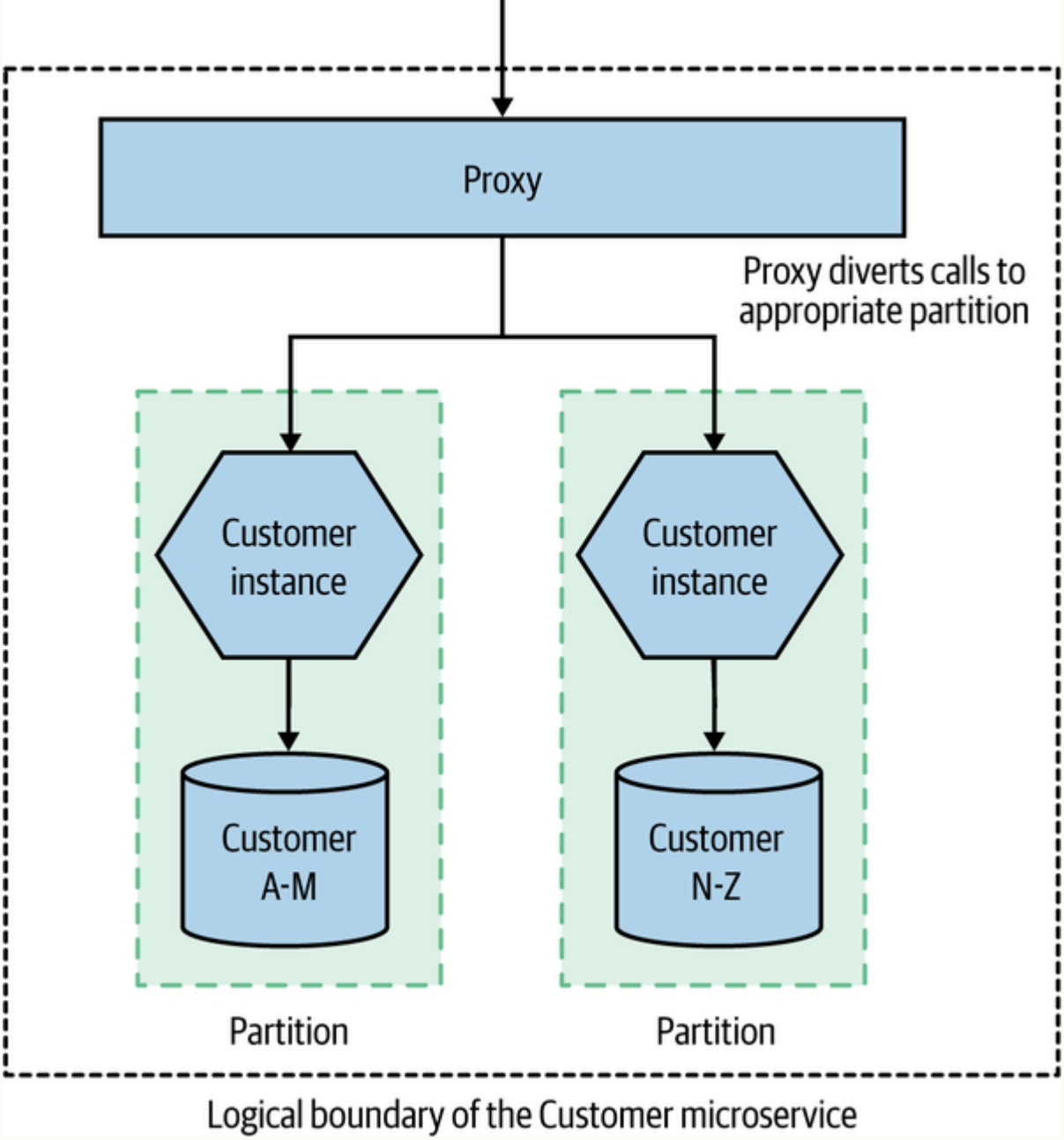


Figure 13-5. Requests are directed to the appropriate microservice instance

As with the example of read replicas, we would want this scaling to be done in such a way that consumers of the microservice are unaware of this implementation detail. When a consumer makes a request to the `Customer` microservice in Figure 13-5, we would want their request to be dynamically routed to the right partition. The fact that we have implemented data partitioning should be treated as an internal implementation detail of the microservice in question—this gives us freedom to change the partitioning scheme, or perhaps replace partitioning altogether.

Another common example of data partitioning is to do it on a geographic basis. You might have one partition per country, or one per region.

For FoodCo, one option to handle the contention on its primary database is to partition data based on country. So customers in Ghana hit one database, and customers in Jersey hit another. This model wouldn't make sense for FoodCo due to a number of factors. The main problem is that FoodCo has plans to continue geographic expansion and is hoping

to drive efficiencies by being able to serve multiple geographical locales from the same system. The idea of having to continually spin up new partitions for each country would drastically increase the cost of moving into new countries.

More often than not, partitioning will be done by the subsystem you rely on. For example, Cassandra uses partitions to distribute both reads and writes across the nodes in a given "ring," and Kafka supports distributing messages across partitioned topics.

Key benefits

Data partitioning scales really nicely for transactional workloads. If your system is write-constrained, for example, data partitioning can deliver huge improvements.

The creation of multiple partitions can also make it easier to reduce the impact and scope of maintenance activities. Rolling out updates can be done on a per-partition basis, and operations that would otherwise require downtime can have reduced impact, as they'll affect only a single partition. For example, if partitioning around geographic regions, operations that might result in interruption of service could be done at the least impactful time of day, perhaps in the early hours of the morning. Geographical partitioning can also be very useful if you need to ensure that data cannot leave certain jurisdictions—making sure that data associated with EU citizens remains stored inside the EU, for example.

Data partitioning can work well with horizontal duplication—each partition could consist of multiple nodes able to handle that work.

Limitations

It's worth pointing out that data partitioning has limited utility in terms of improving system robustness. If a partition fails, that portion of your requests will fail. For example, if your load is distributed evenly across four partitions, and one partition fails, then 25% of your requests will end up failing. This isn't as bad as a total failure, but it's still pretty bad. This is why, as outlined earlier, it is common to combine data partitioning with a technique like horizontal duplication to improve the robustness of a given partition.

Getting the partition key right can be difficult. In Figure 13-5, we used a fairly simple partitioning scheme, where we partitioned workload based on the family name of the customer. Customers with a family name starting with A–M go to partition 1, and customers with a name that starts with N–Z go to partition 2. As I pointed out when I shared

that example, this is not a good partitioning strategy. With data partitioning, we want even distribution of load. But we cannot expect even distribution with the scheme I have outlined. In China, for example, there historically have been a very small number of surnames, and even today there are reckoned to be fewer than 4,000. The most popular 100 surnames, which account for over 80% of the population, skew heavily toward those family names starting with N–Z in Mandarin. This is an example of a scaling scheme that is unlikely to give even distribution of load, and across different countries and cultures could give wildly varying results.

A more sensible alternative might be to partition based on a unique ID given to each customer when they signed up. This is much more likely to give us an even distribution of load and would also deal with the situation in which someone changes their name.

Adding new partitions to an existing scheme can often be done without too much trouble. For example, adding a new node to a Cassandra ring doesn't require any manual rebalancing of the data; instead, Cassandra has built-in support for dynamically distributing data across the nodes. Kafka also makes it fairly easy to add new partitions after the fact, although messages already in a partition won't move—but producers and consumers can be dynamically notified.

Things get more tricky when you realize your partitioning scheme is just not fit for purpose, as in the case of our family name–based scheme outlined earlier. In that situation, you may have a painful road ahead. I remember chatting with a client many years ago who ended up having to take their main production system offline for three days to change the partitioning scheme for their main database.

We can also hit an issue with queries. Looking up an individual record is easy, as I can just apply the hashing function to find which instance the data should be on and then retrieve it from the correct shard. But what about queries that span the data in multiple nodes—for example, finding all the customers who are over 18? If you want to query all shards, you need to either query each individual shard and join in memory or else have an alternative read store where both data sets are available. Often querying across shards is handled by an asynchronous mechanism, using cached results. Mongo uses map/reduce jobs, for example, to perform these queries.

As you may have inferred from this brief overview, scaling databases for writes is where things get very tricky, and where the capabilities of the various databases really start to become differentiated. I often see people changing database technology when they start hitting limits on how easily they can scale their existing write volume. If this happens to you, buying a bigger box is often the quickest way to solve the problem, but in the back-

ground you might want to look at other types of databases that might better handle your requirements. With the wealth of different types of databases available, selecting a new database can be a daunting activity, but as a starting point I thoroughly recommend the pleasingly concise *NoSQL Distilled*,[4] which gives you an overview of the different styles of NoSQL databases available to you—from highly relational stores like graph databases to document stores, column stores, and key-value stores.

Fundamentally, data partitioning is more work, especially as it may likely require extensive changes to the data of your existing system. The application code is probably going to be only lightly impacted, though.

Functional Decomposition

With functional decomposition, you extract functionality and allow it to be scaled independently. Extracting functionality from an existing system and creating a new microservice is almost the canonical example of functional decomposition. In Figure 13-6, we see an example from MusicCorp in which the order functionality is extracted from the main system to allow us to scale this functionality separately from the rest.

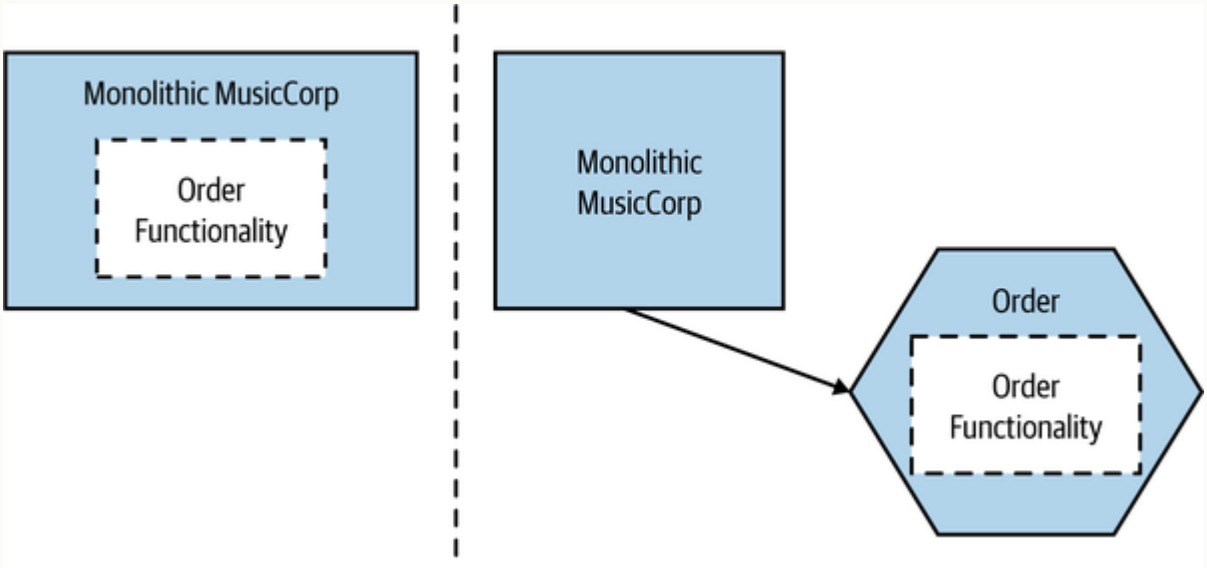


Figure 13-6. The `Order` microservice is extracted from the existing MusicCorp system

In the case of FoodCo, this is its forward direction. The company has exhausted vertical scaling, used horizontal duplication as far as it can, discounted data partitioning—what is left is to start to move toward functional decomposition. Key data and workloads are being removed from the core system and core database to make this change happen. A few quick wins were identified, including data associated with deliveries and menus being moved out of the primary database into dedicated microservices. This has the added benefit that it creates opportunities for FoodCo's growing delivery team to start to organize itself around the ownership of these new microservices.

Implementation

I won't dwell too much on this scaling mechanism, as we've already covered the fundamentals of microservices extensively within this book. For a more detailed discussion of how we can make a change like this happen, see Chapter 3.

Key benefits

The fact that we have split out different types of workloads means we can now rightsize the underlying infrastructure needed for our system. Decomposed functionality that is only occasionally used could be turned off when not needed. Functionality that only has modest load requirements could be deployed onto small machines. On the other hand, functionality that is currently constrained could have more hardware thrown at it, perhaps combining functional decomposition with one of the other scaling axes—such as running multiple copies of our microservice.

This ability to rightsize the infrastructure required to run these workloads gives us more flexibility around optimizing the cost of the infrastructure we need to run the system. This is a key reason why large SaaS providers make such heavy use of microservices, as being able to find the right balance of infrastructure costs can help drive profitability.

By itself, functional decomposition isn't going to make our system more robust, but it at least opens up the opportunity for us to build a system that can tolerate a partial failure of functionality, something we explored in more detail in Chapter 12.

Assuming you've taken the microservices route to functional decomposition, you'll have increased opportunity to use different technologies that can scale the decomposed microservice. For example, you could move the functionality to a programming language and runtime that is more efficient for the type of work you are doing, or perhaps you could migrate the data to a database better suited for your read or write traffic.

Although in this chapter we're focusing primarily on scale in the context of our software system, functional decomposition also makes it easier to scale our organization as well, a topic we'll come back to in Chapter 15.

Limitations

As we explored in detail in Chapter 3, splitting apart functionality can be a complex activity and is unlikely to deliver benefits in the short term. Of all the forms of scaling we've looked at, this is the one that will likely have the biggest impact on your applica-

tion code—on both the frontend and the backend. It can also require a significant amount of work in the data tier if you make the choice to move to microservices as well.

You'll end up increasing the number of microservices you are running, which will increase the overall complexity of the system—potentially leading to more things that need to be maintained, made robust, and scaled. In general, when it comes to scaling a system, I try and exhaust the other possibilities before considering functional decomposition. My view on that may change if the shift to microservices potentially brings with it a host of other things that the organization is looking for. In the case of FoodCo, for example, its drive to grow its development team to both support more countries and deliver more features is key, so a migration toward microservices offers the company a chance to solve not only some of its system scaling issues but also its organizational scaling issues as well.

Combining Models

One of the main drivers behind the original Scale Cube was to stop us from thinking narrowly in terms of one type of scaling, and to help us understand that it often makes sense to scale our application along multiple axes, depending on our need. Let's come back to the example outlined in Figure 13-6. We've extracted our `Order` functionality so it can now run on its own infrastructure. As a next step, we could scale the `Order` microservice in isolation by having multiple copies of it, as we see in Figure 13-7.

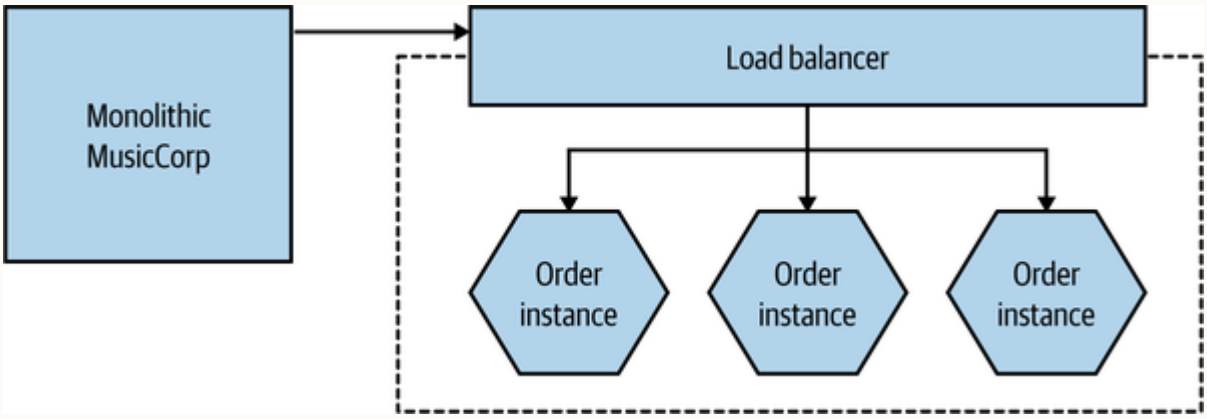


Figure 13-7. The extracted `Order` microservice is now duplicated for scale

Next, we could decide to run different shards of our `Order` microservice for different geographical locales, as in Figure 13-8. Horizontal duplication applies within each geographic boundary.

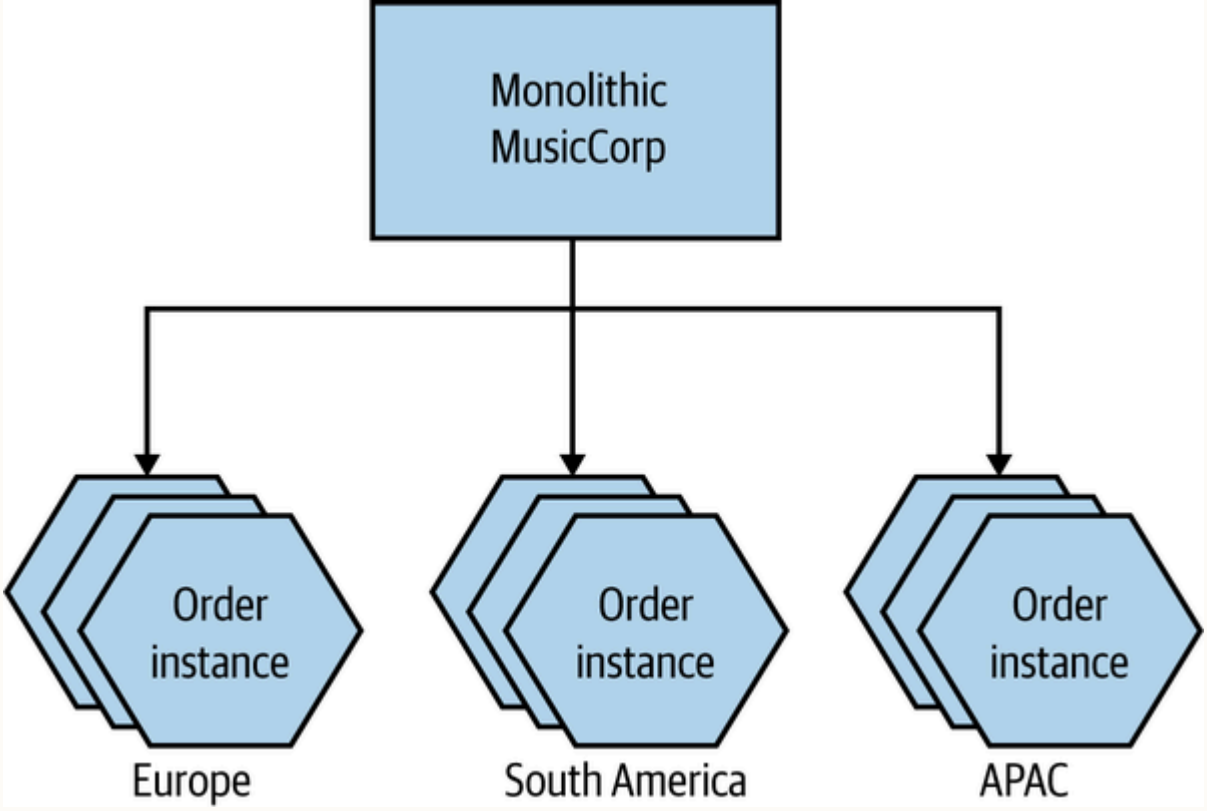


Figure 13-8. MusicCorp's `Order` microservice is now partitioned across geography, with duplication in each group

It's worth noting that by scaling along one axis, other axes might be easier to make use of. For example, the functional decomposition of `Order` enables us to then spin up multiple duplicates of the `Order` microservice, and also to partition the load on order processing. Without that initial functional decomposition, we'd be limited to applying those techniques on the monolith as a whole.

The goal when scaling isn't necessarily to scale along all axes, but we should be aware that we have these different mechanisms at our disposal. Given this choice, it's important we understand the pros and cons of each mechanism to work out which ones make the most sense.

Start Small

In *The Art of Computer Programming* (Addison-Wesley), Donald Knuth famously said:

> *The real problem is that programmers have spent far too much time worrying about efficiency in the wrong places and at the wrong times; premature optimization is the root of all evil (or at least most of it) in programming.*

Optimizing our system to solve problems we don't have is a great way to waste time that could be better spent on other activities, and also to ensure that we have a system that is needlessly more complex. Any form of optimization should be driven by real need. As

we talked about in "Robustness", adding new complexity to our system can introduce new sources of fragility as well. By scaling one part of our application, we create a weakness elsewhere. Our `Order` microservice may now be running on its own infrastructure, helping us better handle the load of the system, but we've got yet another microservice that we need to make sure is available if we want our system to function, and yet more infrastructure that has to be managed and made robust.

Even if you think you have identified a bottleneck, a process of experimentation is essential to make sure that you are right and that further work is justified. It's amazing to me how many people who would happily describe themselves as computer scientists seem not to have even a basic understanding of the scientific method.[5] If you've identified what you think is a problem, try to identify a small amount of work that can be done to confirm whether or not your proposed solution will work. In the context of scaling systems to handle load, having a suite of automated load tests, for example, can be incredibly useful. Run the tests to get a baseline and to re-create the bottleneck you are experiencing, make a change, and observe the difference. This isn't rocket science, but it is, even in a very small way, attempting to be at least vaguely scientific.

CQRS AND EVENT SOURCING

The Command Query Responsibility Segregation (CQRS) pattern refers to an alternate model for storing and querying information. Rather than our having a single model for how we both manipulate and retrieve data, as is common, responsibilities for reads and writes are instead handled by separate models. These separate read and write models, implemented in code, could be deployed as separate units, giving us the ability to scale reads and writes independently. CQRS is often, though not always, used in conjunction with *event sourcing*, where—rather than storing the current state of an entity as a single record—we instead project the state of an entity by looking at the history of events related to that entity.

Arguably, CQRS is doing something very similar in our application tier to what read replicas can do in the data tier, although due to the large number of different ways CQRS can be implemented, this is a simplification.

Personally, although I see value in the CQRS pattern in some situations, it's a complex pattern to execute well. I've spoken to very smart people who have hit not insignificant issues in making CQRS work. As such, if you are considering CQRS as a way to help scale your application, regard it as one of the harder forms of scaling you'd need to implement, and perhaps try some of the easier stuff first. For example, if you are simply read-constrained, a read replica may well be a significantly less risky and faster approach to start with. My concerns about the complexity of implementation extend to event sourcing—there are some situations it fits really well, but it comes with a host of headaches that need to be accommodated. Both patterns require quite a shift in thinking for developers, which always makes things more challenging. If you decide to use either of these patterns, just make sure this increased cognitive load on your developers is worth it.

One final note on CQRS and event sourcing: from the point of view of a microservice architecture, the decision to use or not use these techniques is an internal implementation detail of a microservice. If you've decided to implement a microservice by splitting responsibility for reads and writes across different processes and models, for example, this should be invisible to consumers of the microservice. If inbound requests need to be redirected to the appropriate model based on the request being made, make this the responsibility of the microservice implementing CQRS. Keeping these implementation details hidden from consumers gives you a lot of flexibility to change your mind later, or to change how you are using these patterns.

Caching

Caching is a commonly used performance optimization whereby the previous result of some operation is stored so that subsequent requests can use this stored value rather than spending time and resources recalculating the value.

As an example, consider a `Recommendation` microservice that needs to check stock levels before recommending an item—there isn't any point in recommending something we don't have in stock! But we've decided to keep a local copy of stock levels in `Recommendation` (a form of client-side caching) to improve the latency of our operations—we avoid the need to check stock levels whenever we need to recommend something. The source of truth for stock levels is the `Inventory` microservice, which is considered to be the *origin* for the client cache in the `Recommendation` microservice. When `Recommendation` needs to look up a stock level, it can first look in its local cache. If the entry it needs is found, this is considered a *cache hit*. If the data is not found, it's a *cache miss*, which results in the need to fetch information from the downstream `Inventory` microservice. As the data in the origin can of course change, we need some way to *invalidate* entries in `Recommendation`'s cache so we know when the locally cached data is so out of date that it can no longer be used.

Caches can store the results of simple lookups, as in this example, but really they can store any piece of data, such as the result of a complex calculation. We can cache to help improve the performance of our system as part of helping reduce latency, to scale our application, and in some cases even to improve the robustness of our system. Taken together with the fact that there are a number of invalidation mechanisms we could use, and multiple places where we can cache, it means we have a lot of aspects to discuss when it comes to caching in a microservice architecture. Let's start by talking about what sorts of problems caches can help with.

For Performance

With microservices, we are often concerned about the adverse impact of network latency and about the cost of needing to interact with multiple microservices to get some data. Fetching data from a cache can help greatly here, as we avoid the need for network calls to be made, which also has the impact of reducing load on downstream microservices. Aside from avoiding network hops, it reduces the need to create the data on each request. Consider a situation in which we are asking for the list of the most popular items by genre. This might involve an expensive join query at the database level. We could

cache the results of this query, meaning we'll need to regenerate the results only when the cached data becomes invalidated.

For Scale

If you can divert reads to caches, you can avoid contention on parts of your system to allow it to better scale. An example of this that we've already covered in this chapter is the use of database read replicas. The read traffic is served by the read replicas, reducing the load on the primary database node and allowing reads to be scaled effectively. The reads on a replica are done against data that might be stale. The read replica will eventually get updated by the replication from primary to replica node—this form of cache invalidation is handled automatically by the database technology.

More broadly, caching for scale is useful in any situation in which the origin is a point of contention. Putting caches between clients and the origin can reduce the load on the origin, better allowing it to scale.

For Robustness

If you have an entire set of data available to you in a local cache, you have the potential to operate even if the origin is unavailable—this in turn could improve the robustness of your system. There are a few things to note about caching for robustness. The main thing is that you'd likely need to configure your cache invalidation mechanism to not automatically evict stale data, and to keep data in the cache until it can be updated. Otherwise, as data gets invalidated, it will be removed from the cache, resulting in a cache miss and a failure to get any data, as the origin is unavailable. This means you need to be prepared to read data that could be quite stale if the origin is offline. In some situations this might be fine, while in others it might be highly problematic.

Fundamentally, using a local cache to enable robustness in a situation in which the origin is unavailable means you are favoring availability over consistency.

A technique I saw used at the *Guardian*, and subsequently elsewhere, was to crawl the existing "live" site periodically to generate a static version of the website that could be served in the event of an outage. Although this crawled version wasn't as fresh as the cached content served from the live system, in a pinch it could ensure that a version of the site would get displayed.

Where to Cache

As we have covered multiple times, microservices give you options. And this is absolutely the case with caching. We have lots of different places where we could cache. The different cache locations I'll outline here have different trade-offs, and what sort of optimization you're trying to make will likely point you toward the cache location that makes the most sense for you.

To explore our caching options, let's revisit a situation that we looked at back in "Data Decomposition Concerns", where we were extracting information about sales in MusicCorp. In Figure 13-9, the `Sales` microservice maintains a record of items that have been sold. It tracks only the ID of the item sold and the timestamp of the sale. Occasionally, we want to ask the `Sales` microservice for a list of the top ten best sellers over the previous seven days.

The problem is that the `Sales` microservice doesn't know the names of the records, just the IDs. It's not much use to say, "The best seller this week had the ID 366548, and we sold 35,345 copies!" We want to know the name of the CD with the ID 366548 as well. The `Catalog` microservice stores that information. This means, as Figure 13-9 shows, that when responding to the request for the top ten best sellers, the `Sales` microservice needs to request the names of the top ten IDs. Let's look at how caching might help us and at what types of caches we could use.

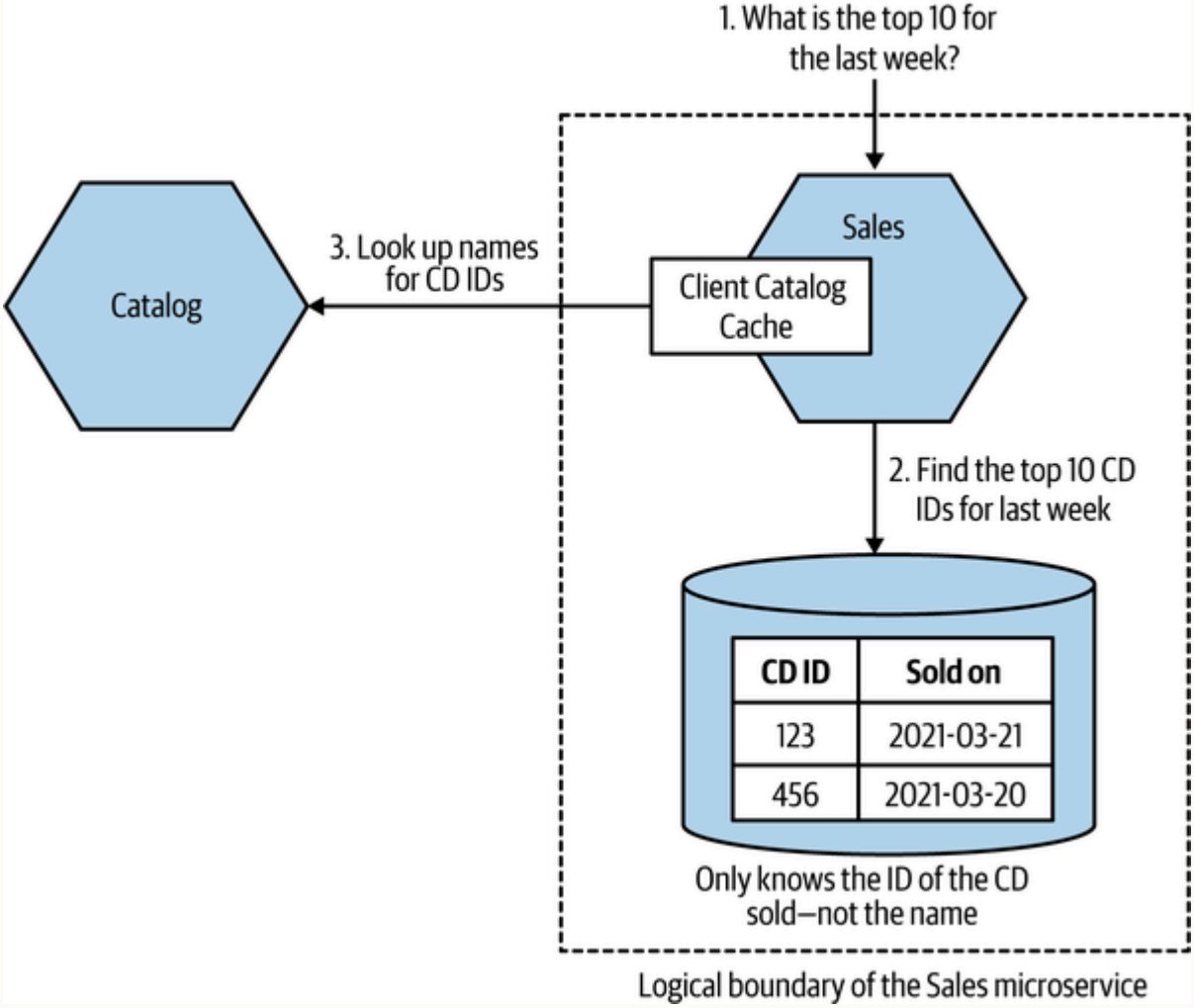


Figure 13-9. An overview of how MusicCorp works out the best sellers

Client-side

With client-side caching, the data is cached outside the scope of the origin. In our example, this could be done as simply as holding an in-memory hashtable with a mapping between ID and name of album inside the running `Sales` process, as in Figure 13-10. This means that generating our top ten takes any interaction with `Catalog` out of scope, assuming we get a cache hit for each lookup we need to make. It's important to note that our client cache could decide to cache only some of the information we get from the microservice. For example, we might get back lots of information about a CD when we ask for information about it, but if all we care about is the name of the album, that's all we have to store in our local cache.

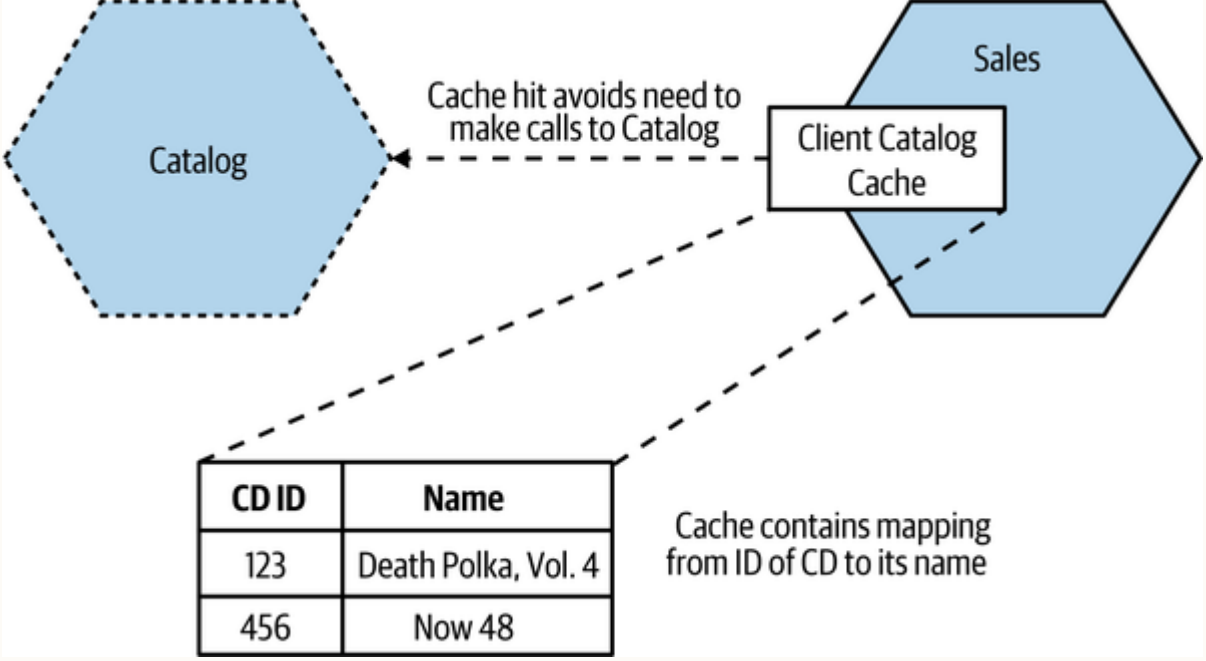


Figure 13-10. `Sales` holds a local copy of `Catalog` data

In general, client-side caches tend to be pretty effective, as they avoid the network call to the downstream microservice. This makes them suitable not only for caching for improved latency but also for caching for robustness.

Client-side caching has a few downsides, though. Firstly, you tend to be more restricted in your options around invalidation mechanisms—something we'll explore more shortly. Secondly, when there's a lot of client-side caching going on, you can see a degree of inconsistency between clients. Consider a situation in which `Sales`, `Recommendation`, and `Promotions` microservices all have a client-side cache of data from `Catalog`. When the data in `Catalog` changes, whatever invalidation mechanism we are likely to use will be unable to guarantee that the data is refreshed at the exact same moment of time in each of those three clients. This means that you could see a different view of the cached data in each of those clients at the same time. The more clients you have, the more problematic this is likely to be. Techniques such as notification-based invalidation, which we'll look at shortly, can help reduce this problem, but they won't eliminate it.

Another mitigation for this is to have a shared client-side cache, perhaps making use of a dedicated caching tool like Redis or memcached, as we see in Figure 13-11. Here, we avoid the problem of inconsistency between the different clients. This can also be more efficient in terms of resource use, as we are reducing the number of copies of this data we need to manage (caches often end up being in memory, and memory is often one of the biggest infrastructural constraints). The flip side is that our clients now need to make a round trip to the shared cache.

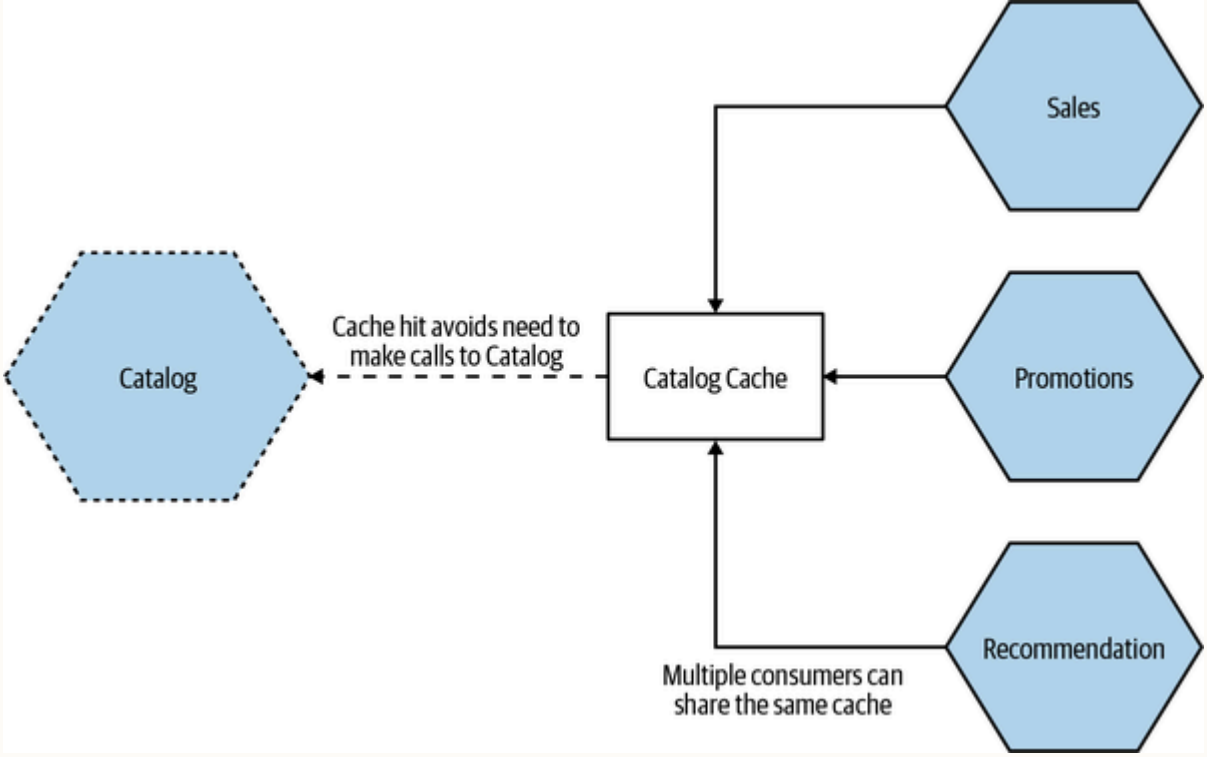


Figure 13-11. Multiple consumers of `Catalog` making use of a single shared cache

Another thing to consider here is who is responsible for this shared cache. Depending on who owns it and how it is implemented, a shared cache like this can blur the lines between client-side caching and server-side caching, which we explore next.

Server-side

In Figure 13-12, we see our top ten sales example making use of caching on the server side. Here, the `Catalog` microservice itself maintains a cache on behalf of its consumers. When the `Sales` microservice makes its request for the names of the CDs, this information is transparently served up by a cache.

Here, the `Catalog` microservice has full responsibility for managing the cache. Due to the nature of how these caches are typically implemented—such as an in-memory data structure, or a local dedicated caching node—it's easier to implement more sophisticated cache invalidation mechanisms. Write-through caches, for example (which we'll look at shortly), would be much simpler to implement in this situation. Having a server-side cache also makes it easier to avoid the issue with different consumers seeing different cached values that can occur with client-side caching.

It's worth noting that, although from the consumer point of view this caching is invisible (it's an internal implementation concern), that doesn't mean we'd have to implement this by caching in code in a microservice instance. We could, for example, maintain a reverse proxy within our microservice's logical boundary, use a hidden Redis node, or divert read queries to read replicas of a database.

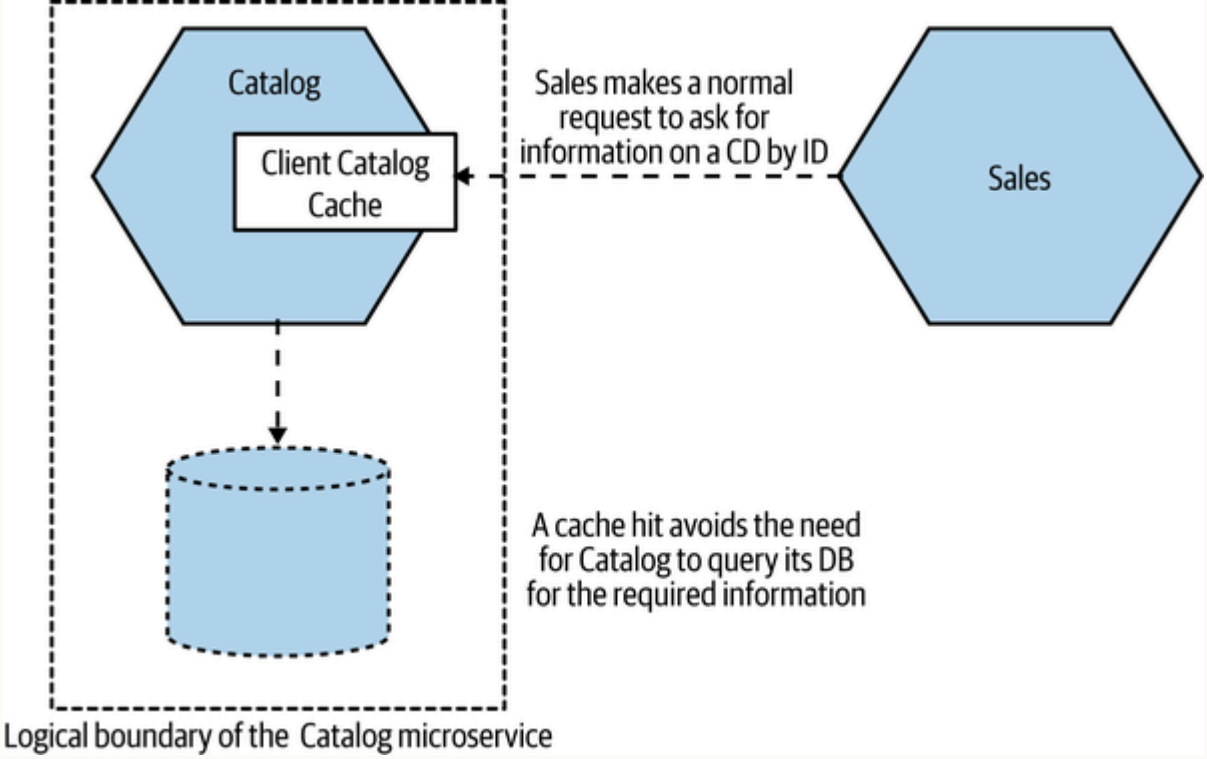


Figure 13-12. The `Catalog` implements caching internally, making it invisible to consumers

The major issue with this form of caching is that it has reduced scope for optimizing for latency, as a round trip by consumers to the microservice is still needed. By caching at or near to the perimeter of a microservice, the cache can ensure that we don't need to carry out further expensive operations (like database queries), but the call needs to be made. This also reduces the effectiveness of this form of caching for any form of robustness.

This might make this form of caching seem less useful, but there is huge value to transparently improving performance for all consumers of a microservice just by making a decision to implement caching internally. A microservice that is widely used across an organization may benefit hugely by implementing some form of internal caching, helping perhaps to improve response times for a number of consumers while also allowing the microservice to scale more effectively.

In the case of our top ten scenario, we'd have to consider whether or not this form of caching might help. Our decision would come down to what our main worry is. If it's about the end-to-end latency of the operation, how much time would a server-side cache save? Client-side caching would likely give us a better performance benefit.

Request cache

With a request cache, we store a cached answer for the original request. So in Figure 13-13 for example, we store the actual top ten entries. Subsequent requests for the top ten best sellers result in the cached result being returned. No lookups in the `Sales` data

needed, no round trips to `Catalog`—this is far and away the most effective cache in terms of optimizing for speed.

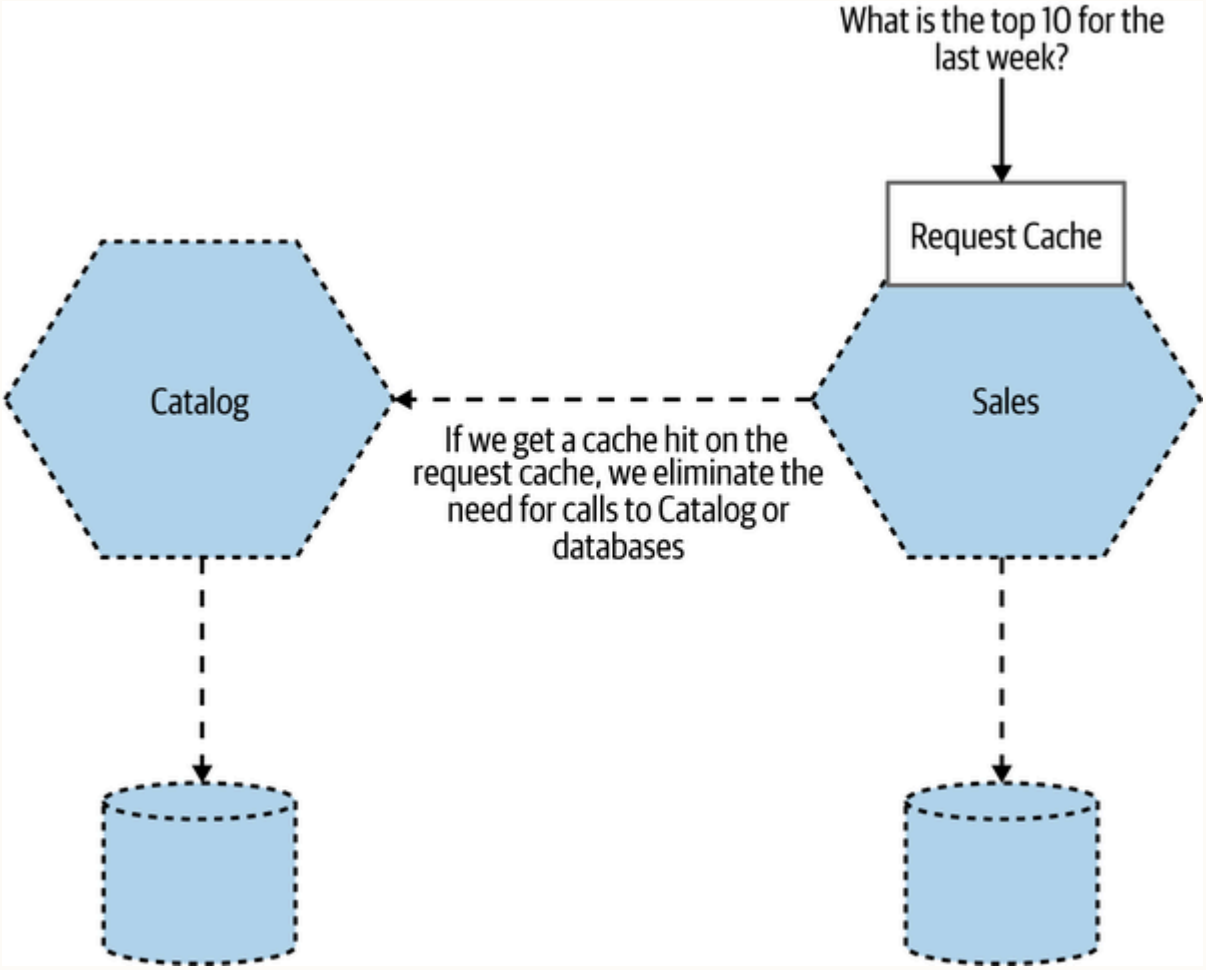


Figure 13-13. Caching the results of the top 10 request

The benefits here are obvious. This is super efficient, for one thing. However, we need to recognize that this form of caching is highly specific. We've only cached the result of this specific request. This means that other operations that hit `Sales` or `Catalog` won't be hitting a cache and thus won't benefit in any way from this form of optimization.

Invalidation

> *There are only two hard things in Computer Science: cache invalidation and naming things.*
>
> —Phil Karlton

Invalidation is the process by which we evict data from our cache. It's an idea that is simple in concept but complex in execution, if for no other reason than there are a wealth of options in terms of how to implement it, and numerous trade-offs to consider in terms of making use of data that might be out of date. Fundamentally, though, it comes down to deciding in which situations a piece of cached data should be removed from your cache.

Sometimes this happens because we are told a new version of a piece of data is available; at other times it might require us to assume our cached copy is stale and fetch a new copy from the origin.

Given the options around invalidation, I think it's a good idea to look at a few of the options you could make use of in a microservice architecture. Please don't consider this to be an exhaustive overview of every option, though!

Time to live (TTL)

This is one of the simplest mechanisms to use for cache invalidation. Each entry in the cache is assumed to be valid for only a certain duration in time. After that time has passed, the data is invalidated, and we fetch a new copy. We can specify the duration of validity using a simple time to live (TTL) duration—so a TTL of five minutes means our cache would happily provide the cache data for up to five minutes, after which the cached entry is considered to be invalidated and a fresh copy is required. Variations on this theme can include using a timestamp for expiration, which in some situations can be more effective, especially if you are reading through multiple levels of cache.

HTTP supports both a TTL (via the `Cache-Control` header) and the ability to set a timestamp for expiration through the `Expires` header on responses, which can be incredibly useful. This means that the origin itself is able to tell downstream clients how long they should assume data is fresh for. Coming back to our `Inventory` microservice, we could imagine a situation in which the `Inventory` microservice gives a shorter TTL for stock levels of fast-selling items, or for items for which we are almost out of stock. For items that we don't sell much of, it could provide a longer TTL. This represents a somewhat advanced use of HTTP cache controls, and tuning cache controls on a per-response basis like this is something I'd do only when tuning the effectiveness of a cache. A simple one-size-fits-all TTL for any given resource type is a sensible starting point.

Even if you're not using HTTP, the idea of the origin giving hints to the client as to how (and if) data should be cached is a really powerful concept. This means you don't have to guess about these things on the client side; you can actually make an informed choice about how to handle a piece of data.

HTTP does have more advanced caching capabilities than this, and we'll look at conditional GETs as an example of that in a moment.

One of the challenges of TTL-based invalidation is that although it is simple to implement, it is a pretty blunt instrument. If we request a fresh copy of the data that has a five-minute TTL, and a second later the data at the origin changes, then our cache will

be operating on out-of-date data for the remaining four minutes and 59 seconds. So the simplicity of implementation needs to be balanced against how much tolerance you have around operating on out-of-date data.

Conditional GETs

Worth a mention, as this is overlooked, is the ability to issue conditional GET requests with HTTP. As we've just touched on, HTTP provides the ability to specify `Cache-Control` and `Expires` headers on responses to enable smarter client-side caching. But if we're working directly with HTTP, we have another option in our arsenal of HTTP goodies: entity tags, or ETags. An ETag is used to determine whether the value of a resource has changed. If I update a customer record, the URI to the resource is the same but the value is different, so I would expect the ETag to change. This becomes powerful when we're using what is called a *conditional GET*. When making a GET request, we can specify additional headers, telling the service to send us the resource only if certain criteria are met.

For example, let's imagine we fetch a customer record, and its ETag comes back as `o5t6fkd2sa`. Later on, perhaps because a `Cache-Control` directive has told us the resource should be considered stale, we want to make sure we get the latest version. When issuing the subsequent GET request, we can pass in an `If-None-Match: o5t6fkd2sa`. This tells the server that we want the resource at the specified URI, unless it already matches this ETag value. If we already have the up-to-date version, the service sends us a `304 Not Modified` response, telling us we have the latest version. If there is a newer version available, we get a `200 OK` with the changed resource and a new ETag for the resource.

Of course, with a conditional GET, we still make the request from client to server. If you are caching to reduce network round trips, this may not help you much. Where it is useful is in avoiding the cost of needlessly regenerating resources. With TTL-based invalidation, the client asks for a new copy of the resource, even if the resource hasn't changed—the microservice receiving this request then has to regenerate that resource, even if it ends up being exactly the same as what the client already has. If the cost of creating the response is high, perhaps requiring an expensive set of database queries, then conditional GET requests can be an effective mechanism.

Notification-based

With notification-based invalidation, we use events to help subscribers know if their local cache entries need to be invalidated. To my mind, this is the most elegant mechanism

for invalidation, though that is balanced by its relative complexity with respect to TTL-based invalidation.

In Figure 13-14, our `Recommendation` microservice is maintaining a client-side cache. Entries in that cache are invalidated when the `Inventory` microservice fires a `Stock Change` event, letting `Recommendation` (or any other subscribers to this event) know that the stock level has increased or decreased for a given item.

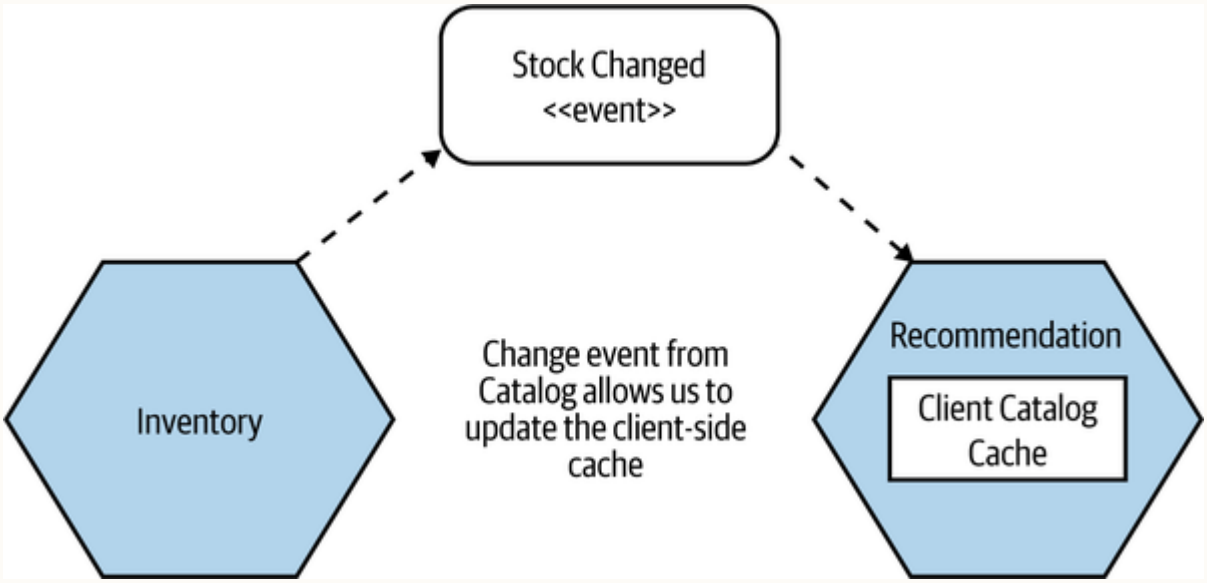


Figure 13-14. `Inventory` fires `Stock Change` events, which `Recommendation` can use to update its local cache

The main benefit of this mechanism is that it reduces the potential window wherein the cache is serving stale data. The window in which a cache might now be serving stale data is limited to the time taken for the notification to be sent and processed. Depending on the mechanism you use to send the notification, this could be pretty fast.

The downside here is the complexity of implementation. We need the origin to be able to emit notifications, and we need interested parties to be able to respond to these notifications. Now, this is a natural place to make use of something like a message broker, as this model fits neatly into the typical pub/sub-style interactions that many brokers provide. The additional guarantees that the broker might be able to give us could also be helpful. That said, as we've already discussed in "Message Brokers", there is an overhead to managing messaging middleware, and it might be overkill if you were using it only for this purpose. If you were using brokers for other forms of inter-microservice communication, however, it would make sense to make use of technology that you already had on hand.

One problem to be aware of when using notification-based invalidation is that you may want to know whether the notification mechanism is actually working or not. Consider a situation in which we haven't received any `Stock Changed` events from `Inventory` for a

while. Does that mean we haven't sold items or had items restocked during that time? Perhaps. It could also mean our notification mechanism is down and that we aren't being sent updates any more. If this is a concern, then we could send a heartbeat event via the same notification mechanism—`Recommendation`, in our case—to let subscribers know that notifications are still coming, but nothing has actually changed. If a heartbeat event isn't received, the client can assume an issue and could do whatever is most appropriate—perhaps informing a user that they are seeing stale data, or perhaps just turning off functionality.

You also need to consider what the notification contains. If the notification just says "this thing has changed" without saying what the change is, then when receiving the notification, a consumer would need to go the origin and fetch the new data. On the other hand, if the notification contains the current state of the data, then consumers can load that directly into their local cache. Having a notification containing more data can cause issues regarding size and also carries a risk of potentially exposing sensitive data too broadly. We previously explored this trade-off in more depth when looking at event-driven communication in "What's in an Event?".

Write-through

With a write-through cache, the cache is updated at the same time as the state in the origin. "At the same time" is where write-through caches get tricky, of course. Implementing a write-through mechanism on a server-side cache is somewhat straightforward, as you could update a database and an in-memory cache within the same transaction without too much difficulty. If the cache is elsewhere, it's more difficult to reason about what "at the same time" means in terms of these entries being updated.

Due to this difficulty, you'd typically see write-through caching being used in a microservice architecture on the server side. The benefits are pretty clear—the window in which a client might see stale data could be practically eliminated. This is balanced against the fact that server-side caches may well be less generally useful, limiting the circumstances in which a write-through cache would be effective in microservices.

Write-behind

With a write-behind cache, the cache itself is updated *first*, and then the origin is updated. Conceptually, you can think of the cache as a buffer. Writing into the cache is faster than updating the origin. So we write the result into the cache, allowing faster subsequent reads, and trust that the origin will be updated afterward.

The main concern around write-behind caches is going to be the potential for data loss. If the cache itself isn't durable, we could lose the data before the data is written to the origin. Additionally, we're now in an interesting spot—what is the *origin* in this context? We'd expect the origin to be the microservice where this data is sourced from—but if we update the cache first, is that really the origin? What is our source of truth? When making use of caching, it's important to separate out what data is cached (and potentially out of date) and what data can actually be considered to be up to date. Write-behind caches in the context of microservices makes this much less clear.

While write-behind caches are often used for in-process optimization, I've seen them much more rarely used for microservice architectures, partly due to the fact that other, more straightforward forms of caching are good enough, but largely due to the complexity of handling the loss of unwritten cached data.

The Golden Rule of Caching

Be careful about caching in too many places! The more caches between you and the source of fresh data, the more stale the data can be, and the harder it can be to determine the freshness of the data that a client eventually sees. It can also be more difficult to reason about where data needs to be invalidated. The trade-off around caching—balancing freshness of data against optimization of your system for load or latency—is a delicate one, and if you cannot easily reason about how fresh (or not) data might be, this becomes difficult.

Consider a situation in which the `Inventory` microservice is caching stock levels. Requests to `Inventory` for stock levels may get served out of this server-side cache, speeding up the request accordingly. Let's also now assume we've set a TTL for this internal cache to be one minute, meaning our server-side cache could be up to one minute behind the actual stock level. Now, it turns out we are also caching on the client side inside `Recommendation`, where we're also using a TTL of one minute. When an entry in the client-side cache expires, we make a request from `Recommendation` to `Inventory` to get an up-to-date stock level, but unbeknownst to us, our request hits the server-side cache, which at this point could also be up to one minute old. So we could end up storing a record in our client-side cache that is already up to one minute old from the start. This means that the stock levels `Recommendation` is using could potentially be up to *two* minutes out of date, even though from the point of view of `Recommendation`, we think they could be only up to one minute out of date.

There are a number of ways to avoid problems like this. Using a timestamp-based expiration for a start would be better than TTLs, but it's also an example of what happens

when caching is effectively nested. If you cache the result of an operation that in turn is based on cached inputs, how clear can you be about how up to date the end result is?

Coming back to the famous quote from Knuth earlier, premature optimization can cause issues. Caching adds complexity, and we want to add as little complexity as possible. The ideal number of places to cache is zero. Anything else should be an optimization you *have* to make—but be aware of the complexity it can bring.

TIP

Treat caching primarily as a performance optimization. Cache in as few places as possible to make it easier to reason about the freshness of data.

Freshness Versus Optimization

Coming back to our example of TTL-based invalidation, I explained earlier that if we request a fresh copy of the data that has a five-minute TTL, and a second later the data at the origin changes, then our cache will be operating on out-of-date data for the remaining four minutes and 59 seconds. If this is unacceptable, one solution would be to reduce the TTL, thereby reducing the duration in which we could operate on stale data. So perhaps we reduce the TTL to one minute. This means that our window of staleness is reduced to one-fifth of what it was, but we've made five times as many calls to the origin, so we have to consider the associated latency and load impact.

Balancing these forces is going to come down to understanding the requirements of the end user and of the wider system. Users will obviously always want to operate on the freshest data, but not if that means the system falls down under load. Likewise, sometimes the safest thing to do is to turn off features if a cache fails, in order to avoid an overload on the origin causing more serious issues. When it comes to fine-tuning what, where, and how to cache, you'll often find yourself having to balance along a number of axes. This is just another reason to try to keep things as simple as possible—the fewer the caches, the easier it can be to reason about the system.

Cache Poisoning: A Cautionary Tale

With caching, we often think that if we get it wrong, the worst thing that can happen is we serve stale data for a bit. But what happens if you end up serving stale data forever? Back in Chapter 12 I introduced AdvertCorp, where I was working to help migrate a number of existing legacy applications over to a new platform making use of the strangler fig

pattern. This involved intercepting calls to multiple legacy applications and, where these applications had been moved to the new platform, diverting the calls. Our new application operated effectively as a proxy. Traffic for the older legacy applications that we hadn't yet migrated was routed through our new application to the downstream legacy applications. For the calls to legacy applications, we did a few housekeeping things; for example, we made sure that the results from the legacy application had proper HTTP cache headers applied.

One day, shortly after a normal routine release, something odd started happening. A bug had been introduced whereby a small subset of pages were falling through a logic condition in our cache header insertion code, resulting in us not changing the header at all. Unfortunately, this downstream application had also been changed sometime previously to include an `Expires: Never` HTTP header. This hadn't had any effect earlier, as we were overriding this header. Now we weren't.

Our application made heavy use of Squid to cache HTTP traffic, and we noticed the problem quite quickly, as we were seeing more requests bypassing Squid itself to hit our application servers. We fixed the cache header code and pushed out a release, and we also manually cleared the relevant region of the Squid cache. However, that wasn't enough.

As we just discussed, you can cache in multiple places—but sometimes having lots of caches makes your life harder, not easier. When it comes to serving up content to users of a public-facing web application, you could have multiple caches between you and your customer. Not only might you be fronting your website with something like a content delivery network, but some ISPs make use of caching. Can you control those caches? And even if you could, there is one cache that you have little control over: the cache in a user's browser.

Those pages with `Expires: Never` stuck in the caches of many of our users and would never be invalidated until the cache became full or the user cleaned them out manually. Clearly we couldn't make either thing happen; our only option was to change the URLs of these pages so they were refetched.

Caching can be very powerful indeed, but you need to understand the full path of data that is cached from source to destination to really appreciate its complexities and what can go wrong.

Autoscaling

If you are lucky enough to have fully automatable provisioning of virtual hosts and can fully automate the deployment of your microservice instances, then you have the building blocks to allow you to automatically scale your microservices.

For example, you could have the scaling triggered by well-known trends. You might know that your system's peak load is between 9 a.m. and 5 p.m., so you bring up additional instances at 8:45 a.m., and turn them off at 5:15 p.m. If you're using something like AWS (which has very good support for autoscaling built in), turning off instances you don't need any longer will help save money. You'll need data to understand how your load changes over time, from day to day, and from week to week. Some businesses have obvious seasonal cycles too, so you may need data going back a fair way to make proper judgment calls.

On the other hand, you could be reactive, bringing up additional instances when you see an increase in load or an instance failure, and removing instances when you no longer need them. Knowing how fast you can scale up once you spot an upward trend is key. If you know you'll get only a couple of minutes' notice about an increase in load, but scaling up will take you at least 10 minutes, then you know you'll need to keep extra capacity around to bridge this gap. Having a good suite of load tests is almost essential here. You can use them to test your autoscaling rules. If you don't have tests that can reproduce different loads that will trigger scaling, then you're only going to find out in production if you got the rules wrong. And the consequences of failure aren't great!

A news site is a great example of a type of business in which you may want a mix of predictive and reactive scaling. On the last news site I worked on, we saw very clear daily trends, with views climbing from the morning to lunchtime and then starting to decline. This pattern was repeated day in and day out, with traffic generally lower on the weekend. That gave us a fairly clear trend that could drive proactive scaling of resources, whether up or down. On the other hand, a big news story would cause an unexpected spike, requiring more capacity and often at short notice.

I actually see autoscaling used much more for handling failure of instances than for reacting to load conditions. AWS lets you specify rules like "There should be at least five instances in this group," so that if one instance goes down a new one is automatically launched. I've seen this approach lead to a fun game of whack-a-mole when someone forgets to turn off the rule and then tries to take down the instances for maintenance, only to see them keep spinning up!

Both reactive and predictive scaling are very useful and can help you be much more cost effective if you're using a platform that allows you to pay only for the computing resources you use. But they also require careful observation of the data available to you. I'd suggest using autoscaling for failure conditions first while you collect the data. Once you want to start autoscaling for load, make sure you are very cautious about scaling down too quickly. In most situations, having more computing power at hand than you need is much better than not having enough!

Starting Again

The architecture that gets you started may not be the architecture that keeps you going when your system has to handle very different volumes of load. As we've already seen, there are some forms of scaling that can have extremely limited impact on the architecture of your system—vertical scaling and horizontal duplication, for example. At certain points, though, you need to do something pretty radical to change the architecture of your system to support the next level of growth.

Recall the story of Gilt, which we touched on in "Isolated Execution". A simple monolithic Rails application did well for Gilt for two years. Its business became increasingly successful, which meant more customers and more load. At a certain tipping point, the company had to redesign the application to handle the load it was seeing.

A redesign may mean splitting apart an existing monolith, as it did for Gilt. Or it might mean picking new data stores that can handle the load better. It could also mean adopting new techniques, such as moving from synchronous request-response to event-based systems, adopting new deployment platforms, changing whole technology stacks, or everything in between.

There is a danger that people will see the need to rearchitect when certain scaling thresholds are reached as a reason to build for massive scale from the beginning. This can be disastrous. At the start of a new project, we often don't know exactly what we want to build, nor do we know if it will be successful. We need to be able to rapidly experiment and understand what capabilities we need to build. If we tried building for massive scale up front, we'd end up front-loading a huge amount of work to prepare for load that may never come, while diverting effort away from more important activities, like understanding if anyone will actually want to use our product. Eric Ries tells the story of spending six months building a product that no one ever downloaded. He reflected that he could have put up a link on a web page that 404'd when people clicked

on it to see if there was any demand, spent six months on the beach instead, and learned just as much!

The need to change our systems to deal with scale isn't a sign of failure. It is a sign of success.

Summary

As we can see, whatever type of scaling you're looking for, microservices give you a lot of different options in terms of how you approach the problem.

The scaling axes can be a useful model to use when considering what types of scaling are available to you:

Vertical scaling

: In a nutshell, this means getting a bigger machine.

Horizontal duplication

: Having multiple things capable of doing the same work.

Data partitioning

: Dividing work based on some attribute of the data, e.g., customer group.

Functional decomposition

: Separation of work based on the type, e.g., microservice decomposition.

Key to a lot of this is understanding what it is you want—techniques that are effective at scaling to improve latency may not be as effective in helping scale for volume.

I hope I have also shown, though, that many of the forms of scaling we have discussed result in increased complexity in your system. So being targeted in terms of what you are trying to change and avoiding the perils of premature optimization are important.

Next up, we move from looking at what happens behind the scenes to the most visible parts of our system—the user interface.

1 Martin L. Abbott and Michael T. Fisher, *The Art of Scalability: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise*, 2nd ed. (New York: Addison-Wesley, 2015).

2 As before, I have anonymized the company—FoodCo isn't its real name!

3 Gregor Hohpe and Bobby Woolf, *Enterprise Integration Patterns* (Boston: Addison-Wesley, 2003).

4 Pramod J. Sadalage and Martin Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence* (Upper Saddle River, NJ: Addison-Wesley 2012).

5 Don't get me started on people who start talking about hypotheses and then go about cherry-picking information to confirm their already-held beliefs.

Previous chapter
< 12. Resiliency

Next chapter
III. People >